Functors, Applicatives, Monads

DM580: Functional Programming

Casper Bach

SDU

Course Plan and Reading

Week 39	Week 40	Week 41	Week 42
Introduction and Haskell Preliminaries *Read:** Foreword + 1 + 2	Types and Basic Functions *Read:* 3 + 4	Lists and Recursion *Read:* 5 + 6 ***Submit handin assn. 1***	*Fall Break*
Week 43	**Week 44**	**Week 45**	**Week 46**
Higher-Order Functions and Type Classes *Read:* 7 + 8 ***Submit handin assn. 2***	Programming with I/O *Read:* 10 ***Submit handin assn. 3***	Monads *Read:* 12	*No lecture* *Read:* TBA ***Submit handin assn. 4***
Week 47	**Week 48**	**Week 49**	**Week 50**
Concurrency *Read:* Beautiful Concurrency	Lazy Evaluation *Read:* 15 ***Submit handin assn. 5***	Exam Project *Read:* TBA	Q&A Session

* Except where otherwise indicated, reading refers to Graham Hutton's *Programming in Haskell*, second edition

Reading Material

This lecture relates to these chapters of *Programming in Haskell, 2nd ed.*

- Chapter 12: Monads and More

Learning Objectives

After following this lecture, you will be able to:

Functors	• Explain what programming pattern *functors* generalize. • Define unary parameterized type constructors, such as lists, `Maybe`, and `Tree`. • Define `fmap` for lists, `Maybe`, and `Tree`.
Applicatives	• Explain what programming pattern *applicatives* generalize • Use `(<*>)` and `pure` for independent effectful computation (Applicative).
Monads	• Explain what programming pattern *monads* generalize • Define `(>>=)` and `pure` for `Maybe` • Translate between **`do`** notation and explicit bind chains.



Motivation

Data Structures as Type Constructors

Data Structures as Type Constructors

- Lists — `data List a = Nil | Cons a (List a)`
  - `List a` is a type. That is, `List a :: *`.
  - We say that `List` is a *type constructor.* It has kind: `List :: * → *`

Data Structures as Type Constructors

- Lists — `data List a = Nil | Cons a (List a)`
  - `List a` is a type. That is, `List a :: *`.
  - We say that `List` is a *type constructor.* It has kind: `List :: * → *`
- Maybe (sometimes known as an *option* type) — `data Maybe a = Nothing | Just a`
  - Type constructor: `Maybe :: * → *`

Data Structures as Type Constructors

- Lists — `data List a = Nil | Cons a (List a)`
  - `List a` is a type. That is, `List a :: *`.
  - We say that `List` is a *type constructor.* It has kind: `List :: * → *`
- Maybe (sometimes known as an *option* type) — `data Maybe a = Nothing | Just a`
  - Type constructor: `Maybe :: * → *`
- Tree — `data Tree a = Leaf | Node (Tree a) a (Tree a)`
  - Type constructor: `Tree :: * → *`

Data Structures as Type Constructors

- Lists — `data List a = Nil | Cons a (List a)`
  - `List a` is a type. That is, `List a :: *`.
  - We say that `List` is a *type constructor.* It has kind: `List :: * → *`
- Maybe (sometimes known as an *option* type) — `data Maybe a = Nothing | Just a`
  - Type constructor: `Maybe :: * → *`
- Tree — `data Tree a = Leaf | Node (Tree a) a (Tree a)`
  - Type constructor: `Tree :: * → *`
- Either — `data Either a b = Left a | Right b`
  - Type constructor: `Either :: * → * → *`
  - Curried type constructor application: `Either a :: * → *`

In Short,

- `data` t `::` $* \to *$
  - Typically a data structure that may contain values of any type.
- Q: What are useful programming patterns for such data structures?
  - Among others, *functors*, *applicatives*, and *monads*

Functors, Applicatives, and Monads in Haskell

Functors, Applicatives, and Monads in Haskell

- Encode *data structures* and classes of operations for these.

Functors, Applicatives, and Monads in Haskell

- Encode *data structures* and classes of operations for these.
- Such structures can emulate side-effects.
  - Brings impurity to Haskell without sacrificing purity.
  - Provides a *denotational semantics* that we can use to reason about imperative side-effects.

Functors, Applicatives, and Monads in Haskell

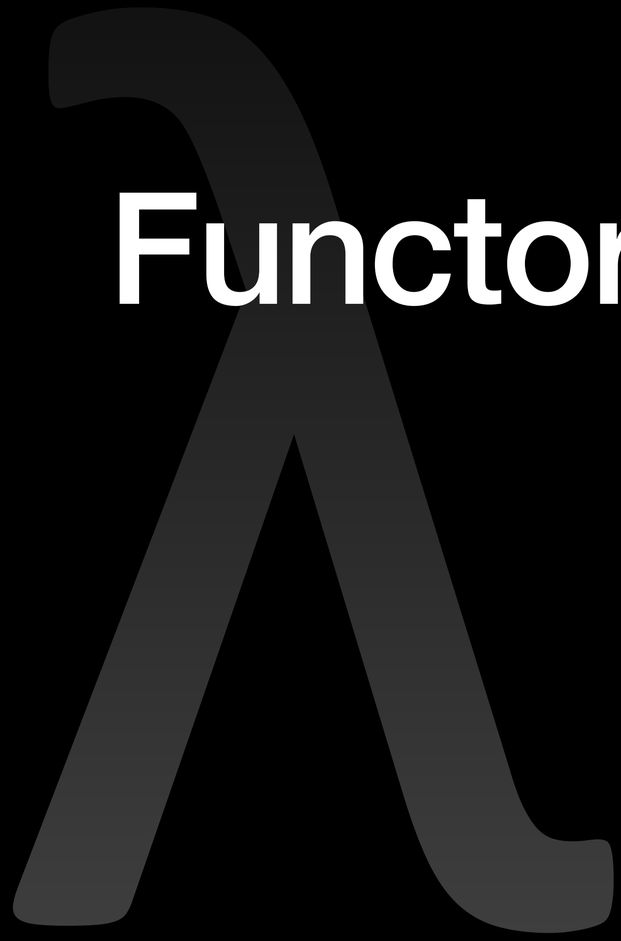
- Encode *data structures* and classes of operations for these.
- Such structures can emulate side-effects.
  - Brings impurity to Haskell without sacrificing purity.
  - Provides a *denotational semantics* that we can use to reason about imperative side-effects.
- *Functor:* Data structure lets us apply functions to contained vals, in structure-preserving way.

Functors, Applicatives, and Monads in Haskell

- Encode *data structures* and classes of operations for these.
- Such structures can emulate side-effects.
  - Brings impurity to Haskell without sacrificing purity.
  - Provides a *denotational semantics* that we can use to reason about imperative side-effects.
- *Functor:* Data structure lets us apply functions to contained vals, in structure-preserving way.
- *Applicative:* Data structures can be combined, independent of contained values.

Functors, Applicatives, and Monads in Haskell

- Encode *data structures* and classes of operations for these.
- Such structures can emulate side-effects.
  - Brings impurity to Haskell without sacrificing purity.
  - Provides a *denotational semantics* that we can use to reason about imperative side-effects.
- *Functor:* Data structure lets us apply functions to contained vals, in structure-preserving way.
- *Applicative:* Data structures can be combined, independent of contained values.
- *Monad:* Data structures can be sequenced, in manner that may depend on contained values.



Functors

Functors

Data structure lets us apply functions to contained values, in structure-preserving way

Functors

Data structure lets us apply functions to contained values, in structure-preserving way

- Lists — `data List a = Nil | Cons a (List a)`
  - ```
    fmap :: (a → b) → List a → List b
    fmap = map
    ```
    ```

Functors

Data structure lets us apply functions to contained values, in structure-preserving way

- Lists — `data List a = Nil | Cons a (List a)`
  - ```
    fmap :: (a → b) → List a → List b
    fmap = map
    ```
- Maybe — `data Maybe a = Nothing | Just a`
  - ```
    fmap :: (a → b) → Maybe a → Maybe b
    fmap _ = _
    ```

Functors

Data structure lets us apply functions to contained values, in structure-preserving way

- Lists — `data List a = Nil | Cons a (List a)`
  - ```
    fmap :: (a → b) → List a → List b
    fmap = map
    ```
- Maybe — `data Maybe a = Nothing | Just a`
  - ```
    fmap :: (a → b) → Maybe a → Maybe b
    fmap _ = _
    ```
- Tree — `data Tree a = Leaf | Node (Tree a) a (Tree a)`
  - ```
    fmap :: (a → b) → Tree a → Tree b
    fmap _ = _
    ```
    ```

Functors

Data structure lets us apply functions to contained values, in structure-preserving way

- Lists — `data List a = Nil | Cons a (List a)`
  - ```
    fmap :: (a → b) → List a → List b
    fmap = map
    ```
- Maybe — `data Maybe a = Nothing | Just a`
  - ```
    fmap :: (a → b) → Maybe a → Maybe b
    fmap _ = _
    ```
- Tree — `data Tree a = Leaf | Node (Tree a) a (Tree a)`
  - ```
    fmap :: (a → b) → Tree a → Tree b
    fmap _ = _
    ```
- Generally — `data t :: * → *`
  - ```
    fmap :: (a → b) → t a → t b
    ```

The `Functor` Type Class

Using Haskell's language extension `{-# LANGUAGE KindSignatures #-}`

The `Functor` Type Class

Using Haskell's language extension `{-# LANGUAGE KindSignatures #-}`

- ```
  class Functor (t :: * → *) where
    fmap :: (a → b) → t a → t b
  ```
  ```

The `Functor` Type Class

Using Haskell's language extension `{-# LANGUAGE KindSignatures #-}`

- ```
  class Functor (t :: * → *) where
    fmap :: (a → b) → t a → t b
  ```

- ```
  instance Functor Maybe where
    fmap _ Nothing  = Nothing
    fmap f (Just x) = Just (f x)
  ```

The `Functor` Type Class

Using Haskell's language extension `{-# LANGUAGE KindSignatures #-}`

- ```
  class Functor (t :: * → *) where
    fmap :: (a → b) → t a → t b
  ```

- ```
  instance Functor Maybe where
    fmap _ Nothing  = Nothing
    fmap f (Just x) = Just (f x)
  ```

- ```
  instance Functor Tree where
    fmap _ Leaf          = Leaf
    fmap f (Node l x r) = Node (fmap f l) (f x) (fmap f r)
  ```
  ```

The `Functor` Type Class

Using Haskell's language extension `{-# LANGUAGE KindSignatures #-}`

- ```
  class Functor (t :: * → *) where
    fmap :: (a → b) → t a → t b
  ```

- ```
  instance Functor Maybe where
    fmap _ Nothing  = Nothing
    fmap f (Just x) = Just (f x)
  ```

- ```
  instance Functor Tree where
    fmap _ Leaf          = Leaf
    fmap f (Node l x r) = Node (fmap f l) (f x) (fmap f r)
  ```

- Etc.
  ```

Non-Example of a Functor

Non-Example of a Functor

- ```
  instance Functor List where
    fmap f _ = Nil
  ```
  ```

Non-Example of a Functor

- ```
  instance Functor List where
    fmap f _ = Nil
  ```
- Well-typed but wrong.
  ```

Non-Example of a Functor

- ```
  instance Functor List where
    fmap f _ = Nil
  ```
- Well-typed but wrong.
- Functor instances are morally obliged to obey these laws.
  - `fmap id x == x`
  - `fmap g (fmap f x) == fmap (g . f) x`
  ```

Non-Example of a Functor

- ```
  instance Functor List where
    fmap f _ = Nil
  ```
- Well-typed but wrong.
- Functor instances are morally obliged to obey these laws.
  - `fmap id x == x`
  - `fmap g (fmap f x) == fmap (g . f) x`
- The instance above does not obey the first law, so it is not a valid functor.
  ```

Deriving Functor

Deriving Functor

- Haskell can typically derive functor instances for us.

Deriving Functor

- Haskell can typically derive functor instances for us.
- `data List a = Nil | Cons a (List a) deriving Functor`

Deriving Functor

- Haskell can typically derive functor instances for us.
- `data List a = Nil | Cons a (List a) deriving Functor`
  - Provides the expected fmap (i.e., standard map) function for lists.

Deriving Functor

- Haskell can typically derive functor instances for us.
- `data List a = Nil | Cons a (List a) deriving Functor`
  - Provides the expected fmap (i.e., standard map) function for lists.
- Similarly, for more exotic data types. E.g.,

Deriving Functor

- Haskell can typically derive functor instances for us.
- `data List a = Nil | Cons a (List a) deriving Functor`
  - Provides the expected fmap (i.e., standard map) function for lists.
- Similarly, for more exotic data types. E.g.,
- ```
  data EitherF (f :: * → *) (g :: * → *) a
    = LeftF (f a)
    | RightF (g a)
    deriving Functor
  ```
  ```

Deriving Functor

- Haskell can typically derive functor instances for us.
- `data List a = Nil | Cons a (List a) deriving Functor`
  - Provides the expected fmap (i.e., standard map) function for lists.
- Similarly, for more exotic data types. E.g.,
- ```
  data EitherF (f :: * → *) (g :: * → *) a
    = LeftF (f a)
    | RightF (g a)
    deriving Functor
  ```
  - Provides
    ```
    instance (Functor f, Functor g) ⇒ Functor (EitherF f g) where
      fmap f (LeftF x)  = LeftF (f x)
      fmap f (RightF x) = RightF (g x)
    ```
    ```

Learning Objectives

After following this lecture, you will be able to:

Functors	✓ Explain what programming pattern *functors* generalize. ✓ Define unary parameterized type constructors, such as lists, `Maybe`, and `Tree`. ✓ Define `fmap` for lists, `Maybe`, and `Tree`.
Applicatives	• Explain what programming pattern *applicatives* generalize • Use `(<*>)` and `pure` for independent effectful computation (Applicative).
Monads	• Explain what programming pattern *monads* generalize • Define `(>>=)` and `pure` for `Maybe` • Translate between **`do`** notation and explicit bind chains.



Applicatives

Applicative

Data structures can be combined, independent of contained values

Applicative

Data structures can be combined, independent of contained values

- Maybe — `data Maybe a = Nothing | Just a`

Applicative

Data structures can be combined, independent of contained values

- Maybe — `data Maybe a = Nothing | Just a`
  - Say we want to combine values contained in Maybe's.

Applicative

Data structures can be combined, independent of contained values

- Maybe — `data Maybe a = Nothing | Just a`
  - Say we want to combine values contained in Maybe's.

```
combine :: Int → Int → Int
```

Applicative

Data structures can be combined, independent of contained values

- Maybe — `data Maybe a = Nothing | Just a`
  - Say we want to combine values contained in Maybe's.

```
combine :: Int → Int → Int

combineMaybe :: Maybe Int → Maybe Int → Maybe Int
combineMaybe m1 m2 =
```

Applicative

Data structures can be combined, independent of contained values

- Maybe — `data Maybe a = Nothing | Just a`
  - Say we want to combine values contained in Maybe's.

```
combine :: Int → Int → Int

combineMaybe :: Maybe Int → Maybe Int → Maybe Int
combineMaybe m1 m2 =
  let f :: Maybe (Int → Int)
      f = fmap combine m1
  in _ -- how to apply f :: Maybe (Int → Int) to m2 :: Maybe Int??
```

Applicative

Data structures can be combined, independent of contained values

- Maybe — `data Maybe a = Nothing | Just a`
  - Say we want to combine values contained in Maybe's.

```
combine :: Int → Int → Int

combineMaybe :: Maybe Int → Maybe Int → Maybe Int
combineMaybe m1 m2 =
  let f :: Maybe (Int → Int)
      f = fmap combine m1
  in _ -- how to apply f :: Maybe (Int → Int) to m2 :: Maybe Int??
```

  - Q: how do we apply a function in-context? A: *Applicatives*.

Applicative

Data structures can be combined, independent of contained values

- Maybe — `data Maybe a = Nothing | Just a`
  - Say we want to combine values contained in Maybe's.

```
combine :: Int → Int → Int

combineMaybe :: Maybe Int → Maybe Int → Maybe Int
combineMaybe m1 m2 =
  let f :: Maybe (Int → Int)
      f = fmap combine m1
  in _ -- how to apply f :: Maybe (Int → Int) to m2 :: Maybe Int??
```

  - Q: how do we apply a function in-context? A: *Applicatives*.

```
(<*>) :: Maybe (a → b) → Maybe a → Maybe b  -- Applicative apply
Just f  <*> Just x  = Just (f x)
Nothing <*> _       = Nothing
_       <*> Nothing = Nothing
```

Applicative

Data structures can be combined, independent of contained values

- Maybe — `data Maybe a = Nothing | Just a`
  - Say we want to combine values contained in Maybe's.

```
combine :: Int → Int → Int

combineMaybe :: Maybe Int → Maybe Int → Maybe Int
combineMaybe m1 m2 =
  let f :: Maybe (Int → Int)
      f = fmap combine m1
  in f <*> m2
```

  - Q: how do we apply a function in-context? A: *Applicatives*.

```
(<*>) :: Maybe (a → b) → Maybe a → Maybe b  -- Applicative apply
Just f  <*> Just x  = Just (f x)
Nothing <*> _       = Nothing
_       <*> Nothing = Nothing
```

The Applicative Type Class

The Applicative Type Class

- ```
  class Functor t ⇒ Applicative (t :: * → *) where
    pure  :: a → t a                      -- Applicative unit
    (<*>) :: t (a → b) → t a → t b        -- Applicative apply
  ```
  ```

The Applicative Type Class

- ```
  class Functor t ⇒ Applicative (t :: * → *) where
    pure  :: a → t a                      -- Applicative unit
    (<*>) :: t (a → b) → t a → t b        -- Applicative apply
  ```
- Applicatives “combine” structures (effects) via function application, but generalized to functions and values inside a structure
  ```

The Applicative Type Class

- ```
  class Functor t ⇒ Applicative (t :: * → *) where
    pure  :: a → t a                      -- Applicative unit
    (<*>) :: t (a → b) → t a → t b        -- Applicative apply
  ```
- Applicatives “combine” structures (effects) via function application, but generalized to functions and values inside a structure
- `pure` mediates pure values into a context (computation).
  ```

The Applicative Type Class

- ```
  class Functor t ⇒ Applicative (t :: * → *) where
    pure  :: a → t a                      -- Applicative unit
    (<*>) :: t (a → b) → t a → t b        -- Applicative apply
  ```
- Applicatives “combine” structures (effects) via function application, but generalized to functions and values inside a structure
- `pure` mediates pure values into a context (computation).
- Applicative apply `(<*>)` applies functions, and nests to the left like regular function application.
  - Let `f :: t (a → b → c)`, `x :: a`, `y :: b`,
  - ```
    f <*> x <*> y        :: f c
    = (f <*> x) <*> y    :: f c
    ```

Applicatives for Effects

Applicatives for Effects

- We can use applicatives to combine effectful computations.

Applicatives for Effects

- We can use applicatives to combine effectful computations.
- For `Maybe`, it represents combining possibly-abruptly terminating computations: if any argument fails, everything fails.

Applicatives for Effects

- We can use applicatives to combine effectful computations.
- For `Maybe`, it represents combining possibly-abruptly terminating computations: if any argument fails, everything fails.
- For lists, it (typically) combining arguments non-deterministically (next slide)

Applicatives for Effects

- We can use applicatives to combine effectful computations.
- For `Maybe`, it represents combining possibly-abruptly terminating computations: if any argument fails, everything fails.
- For lists, it (typically) combining arguments non-deterministically (next slide)
- Trees do not have an obvious interpretation as effects, but can nevertheless be given a zip-like semantics.

Applicatives for Effects

- We can use applicatives to combine effectful computations.
- For `Maybe`, it represents combining possibly-abruptly terminating computations: if any argument fails, everything fails.
- For lists, it (typically) combining arguments non-deterministically (next slide)
- Trees do not have an obvious interpretation as effects, but can nevertheless be given a zip-like semantics.
- Many other functors do have applicative instances. E.g.,
  - `Either a`, `IO`, `(→) r`, and more.

Applicatives for Lists

Applicatives for Lists

- *Cartesian-product*-like semantics:

```
instance Applicative [] where
  pure x = [x]
  fs <*> xs = [f x | f ← fs, x ← xs]
```

Applicatives for Lists

- *Cartesian-product*-like semantics:

```
instance Applicative [] where
  pure x = [x]
  fs <*> xs = [f x | f ← fs, x ← xs]
```

- Intuition: applicative apply represents a *non-deterministic choice* between all possible ways we could possibly combine a function and a value. We enumerate them all. Try, e.g.,
  - ```
    prods :: [Int] → [Int] → [Int]
    prods xs ys = pure (*) <*> xs <*> ys
    ```
  - ```
    prods (pure 1 ++ pure 2) (pure 3 ++ pure 4)
    ```

Applicatives for Lists

- *Cartesian-product*-like semantics:

```
instance Applicative [] where
  pure x = [x]
  fs <*> xs = [f x | f ← fs, x ← xs]
```

- Intuition: applicative apply represents a *non-deterministic choice* between all possible ways we could possibly combine a function and a value. We enumerate them all. Try, e.g.,
  - ```
    prods :: [Int] → [Int] → [Int]
    prods xs ys = pure (*) <*> xs <*> ys
    ```
  - ```
    prods (pure 1 ++ pure 2) (pure 3 ++ pure 4)
    ```

- *Zip*-like semantics of applicative apply is also possible; see, e.g.,

https://hackage-content.haskell.org/package/ghc-internal-9.1401.0/docs/src/GHC.Internal.Functor.ZipList.html#line-68

The Applicative Laws

- The laws that characterize applicatives are provided in the book, just before Chapter 12.3.

- These laws hold for both applicative instances on the previous slide.

Learning Objectives

After following this lecture, you will be able to:

Functors	✓ Explain what programming pattern *functors* generalize. ✓ Define unary parameterized type constructors, such as lists, `Maybe`, and `Tree`. ✓ Define `fmap` for lists, `Maybe`, and `Tree`.
Applicatives	✓ Explain what programming pattern *applicatives* generalize ✓ Use `(<*>)` and `pure` for independent effectful computation (Applicative).
Monads	• Explain what programming pattern *monads* generalize • Define `(>>=)` and `pure` for `Maybe` • Translate between **`do`** notation and explicit bind chains.

Monads



Monads

Data structures can be sequenced, in manner that may depend on contained values

Monads

Data structures can be sequenced, in manner that may depend on contained values

- Applicatives let us combine effects in ways that assumes the effects can be run in parallel, and their values combined independently of effects.

Monads

Data structures can be sequenced, in manner that may depend on contained values

- Applicatives let us combine effects in ways that assumes the effects can be run in parallel, and their values combined independently of effects.
- Sometimes we want to write programs where what the “rest” of the computation should be depends on the value produced. E.g.,
  - ```
    data Either x y = Left x | Right x deriving (Functor, Show, Eq)

    readAndProceedIfGt0 :: IO x → IO (Either String x)
    readAndProceedIfGt0 m = do
      ns ← getLine
      if (read ns :: Integer) > 0
        then fmap Right m
        else pure (Left "The number was not greater than zero.”)
    ```
    ```

Monads

Data structures can be sequenced, in manner that may depend on contained values

- Applicatives let us combine effects in ways that assumes the effects can be run in parallel, and their values combined independently of effects.
- Sometimes we want to write programs where what the “rest” of the computation should be depends on the value produced. E.g.,
  - ```
    data Either x y = Left x | Right x deriving (Functor, Show, Eq)

    readAndProceedIfGt0 :: IO x → IO (Either String x)
    readAndProceedIfGt0 m = do
      ns ← getLine
      if (read ns :: Integer) > 0
        then fmap Right m
        else pure (Left "The number was not greater than zero.”)
    ```
- We cannot run `getLine` and the if-then-else expression in parallel: the if-then-else depends on the value yielded by `getLine`. We need a more specific notion of computation: *monads*.
    ```

Sequencing with Monadic Bind

Sequencing with Monadic Bind

- ```
  class Applicative t ⇒ Monad t where
    (>>=) :: t a → (a → t b) → t b
  ```
  ```

Sequencing with Monadic Bind

- `class Applicative t ⇒ Monad t where`
  `(>>=) :: t a → (a → t b) → t b`

- **do** notation is “just” syntactic sugar for nested applications of monadic bind.

  ▸
```
do x1 ← m1
   x2 ← m2
   ⋮
   xn ← mn
   f x1 x2 … xn
```

```
m1 >>= \x1 →
m2 >>= \x2 →
⋮
mn >>= \xn →
f x1 x2 … xn
```

Sequencing with Monadic Bind

- ```
  class Applicative t ⇒ Monad t where
    (>>=) :: t a → (a → t b) → t b
  ```

- **do** notation is “just” syntactic sugar for nested applications of monadic bind.

  ▸
  ```
  do x1 ← m1
     x2 ← m2
     ⋮
     xn ← mn
     f x1 x2 … xn
  ```

  ```
  m1 >>= \x1 →
  m2 >>= \x2 →
  ⋮
  mn >>= \xn →
  f x1 x2 … xn
  ```

- Let’s illustrate the use of monadic bind by revisiting the arithmetic expression interpreter from the lecture slides for “Declaring Types and Classes”.
  ```

Revisiting our Arithmetic Interpreter

Revisiting our Arithmetic Interpreter

- `data Expr = Val Int | Add Expr Expr | Div Expr Expr`

Revisiting our Arithmetic Interpreter

- `data Expr = Val Int | Add Expr Expr | Div Expr Expr`
- ```
  eval0 :: Expr → Int
  eval0 (Val i) = i
  eval0 (Add e1 e2) = eval0 e1 + eval0 e2
  eval0 (Div e1 e2) = eval0 e1 `div` eval0 e2
  ```
  ```

Revisiting our Arithmetic Interpreter

- `data Expr = Val Int | Add Expr Expr | Div Expr Expr`
- ```
  eval0 :: Expr → Int
  eval0 (Val i) = i
  eval0 (Add e1 e2) = eval0 e1 + eval0 e2
  eval0 (Div e1 e2) = eval0 e1 `div` eval0 e2
  ```
- Not a great semantics, since

  ```
  eval0 (Val 1 `Div` Val 0)
  ```

  crashes the run time and we cannot recover from the error.
  ```

Adding an Effect, Suboptimally

Adding an Effect, Suboptimally

- `data Expr = Val Int | Add Expr Expr | Div Expr Expr`

Adding an Effect, Suboptimally

- `data Expr = Val Int | Add Expr Expr | Div Expr Expr`

- ```
  eval1 :: Expr → Maybe Int
  eval1 (Val i) = Just i
  eval1 (Add e1 e2) = case eval1 e1 of
    Just n → case eval1 e2 of
      Just m → Just (m + n)
      Nothing → Nothing
    Nothing → Nothing
  eval1 (Div e1 e2) = -- elided
  ```
  ```

Adding an Effect, Suboptimally

- `data Expr = Val Int | Add Expr Expr | Div Expr Expr`
- ```
  eval1 :: Expr → Maybe Int
  eval1 (Val i) = Just i
  eval1 (Add e1 e2) = case eval1 e1 of
    Just n → case eval1 e2 of
      Just m → Just (m + n)
      Nothing → Nothing
    Nothing → Nothing
  eval1 (Div e1 e2) = -- elided
  ```
- We've extended our interpreter, but at the cost of rewriting all existing cases, and making it difficult to read. 👎
  ```

Adding an effect, using Monadic Bind

A better semantics

Adding an effect, using Monadic Bind

A better semantics

- ```
  instance Monad Maybe where -- (>>=) :: Maybe a → (a → Maybe b) → Maybe b
    Just x  >>= k = k x
    Nothing >>= _ = Nothing
  ```
  ```

Adding an effect, using Monadic Bind

A better semantics

- ```
  instance Monad Maybe where -- (>>=) :: Maybe a → (a → Maybe b) → Maybe b
    Just x  >>= k = k x
    Nothing >>= _ = Nothing
  ```

- ```
  safediv :: Int → Int → Maybe Int
  safediv n m | m /= 0    = Just (n `div` m)
              | otherwise = Nothing
  ```

Adding an effect, using Monadic Bind

A better semantics

- ```
  instance Monad Maybe where -- (>>=) :: Maybe a → (a → Maybe b) → Maybe b
    Just x  >>= k = k x
    Nothing >>= _ = Nothing
  ```

- ```
  safediv :: Int → Int → Maybe Int
  safediv n m | m /= 0    = Just (n `div` m)
              | otherwise = Nothing
  ```

- ```
  eval2 :: Expr → Maybe Int
  eval2 (Num i) = pure i
  eval2 (Add e1 e2) =
    eval2 e1 >>= \n →
    eval2 e2 >>= \m →
    pure (n + m)
  eval2 (Div e1 e2) =
    eval2 e1 >>= \n →
    eval2 e2 >>= \m →
    safediv n m
  ```
  ```

Would This Work?

- ```
  safediv :: Int → Int → Maybe Int
  safediv n m | m /= 0    = Just (n `div` m)
              | otherwise = Nothing
  ```

- ```
  eval2 :: Expr → Maybe Int
  eval2 (Add e1 e2) =
    pure (+) <*> eval2 e1 <*> eval2 e2
  {- … -}
  ```

Would This Work?

- ```
  safediv :: Int → Int → Maybe Int
  safediv n m | m /= 0    = Just (n `div` m)
              | otherwise = Nothing
  ```

- ```
  eval2 :: Expr → Maybe Int
  {- … -}
  eval2 (Div e1 e2) =
    pure safediv <*> eval2 e1 <*> eval2 e2
  ```

Extending Our Interpreter without Altering Cases

But altering how effects are defined.

Extending Our Interpreter without Altering Cases

But altering how effects are defined.

- `data Expr = Val Int | Add Expr Expr | Div Expr Expr | Trace String Expr`

Extending Our Interpreter without Altering Cases

But altering how effects are defined.

- `data Expr = Val Int | Add Expr Expr | Div Expr Expr | Trace String Expr`
- `newtype Computation a = C { runC :: (Maybe a, [String]) } deriving Functor`

Extending Our Interpreter without Altering Cases

But altering how effects are defined.

- `data Expr = Val Int | Add Expr Expr | Div Expr Expr | Trace String Expr`
- `newtype Computation a = C { runC :: (Maybe a, [String]) } deriving Functor`
- ```
  instance Applicative Computation where -- ??
  instance Applicative Monad where -- ??
  ```
  ```

Extending Our Interpreter without Altering Cases

But altering how effects are defined.

- `data Expr = Val Int | Add Expr Expr | Div Expr Expr | Trace String Expr`
- `newtype Computation a = C { runC :: (Maybe a, [String]) } deriving Functor`
- ```
  instance Applicative Computation where -- ??
  instance Applicative Monad where -- ??
  ```
- ```
  trace :: String → Computation ()
  trace s = C (Just (), [s])
  ```

Extending Our Interpreter without Altering Cases

But altering how effects are defined.

- `data Expr = Val Int | Add Expr Expr | Div Expr Expr | Trace String Expr`
- `newtype Computation a = C { runC :: (Maybe a, [String]) } deriving Functor`
- ```
  instance Applicative Computation where -- ??
  instance Applicative Monad where -- ??
  ```
- ```
  trace :: String → Computation ()
  trace s = C (Just (), [s])
  ```
- ```
  eval :: Expr → Computation Int
  eval (Num i) = pure i -- unchanged!
  eval (Add e1 e2) =    -- unchanged!
    eval e1 >>= \n →
    eval e2 >>= \m →
    pure (n + m)
  eval (Div e1 e2) =    -- unchanged!
    eval e1 >>= \n →
    eval e2 >>= \m →
    safediv n m
  eval (Trace str e) =
    trace str >>= _ → eval e
  ```
  ```

Monad Laws and More Examples

- The book covers more examples, in an accessible manner.
- It also summarizes the following laws, which explain how monadic sequencing behaves.
  - `pure x >>= f  =  f x`
  - `m >>= pure  =  m`
  - `(m >>= f) >>= g  =  m >>= (\x -> (f x >>= g))`

Learning Objectives

After following this lecture, you will be able to:

Functors	✓ Explain what programming pattern *functors* generalize. ✓ Define unary parameterized type constructors, such as lists, `Maybe`, and `Tree`. ✓ Define `fmap` for lists, `Maybe`, and `Tree`.
Applicatives	✓ Explain what programming pattern *applicatives* generalize ✓ Use `(<*>)` and `pure` for independent effectful computation (Applicative).
Monads	✓ Explain what programming pattern *monads* generalize ✓ Define `(>>=)` and `pure` for `Maybe` ✓ Translate between **`do`** notation and explicit bind chains.